

Question 2. Scalable Logistic Regression and Generalized Linear Models [10 marks, set by Shuo Zhou]

Please carefully read the [assignment brief](#) before starting to complete the assignment.

Prerequisites:

Before attempting this question, you are expected to have completed Lab 3 and Lab 4.

Data:

You are given a UK traffic vehicle-count dataset containing over 5 million observations collected from 43,105 counting points from 2000 to 2024. Each record corresponds to the number of vehicles recorded at a traffic counting point during a specific hour.

This dataset has been uploaded to Stanage and is available at:

Shell

```
/mnt/parscratch/users/com6012_2026/data/dft_traffic_counts_raw_counts.csv
```

You have read access to this directory, but you do not have write permission.

For more information about the dataset, see the government website:

<https://roadtraffic.dft.gov.uk/downloads#road-traffic-data-guidance>.

Task:

Use 10 cores and 10 GB of memory per core for the following tasks. The entire program should take approximately 1 hour to complete, depending on your implementation. Cache intermediate DataFrames when appropriate to avoid recomputing expensive transformations.

A. Load and process the dataset

Load the dataset from the CSV file into a Spark DataFrame. The dataset contains the following columns:

Python

```
['count_point_id', 'direction_of_travel', 'year', 'count_date', 'hour',  
'region_id', 'region_name', 'region_ons_code', 'local_authority_id',  
'local_authority_name', 'local_authority_code', 'road_name',  
'road_category', 'road_type', 'start_junction_road_name',  
'end_junction_road_name', 'easting', 'northing', 'latitude',  
'longitude', 'link_length_km', 'link_length_miles', 'pedal_cycles',  
'two_wheeled_motor_vehicles', 'cars_and_taxis', 'buses_and_coaches',  
'LGVs', 'HGVs_2_rigid_axle', 'HGVs_3_rigid_axle',  
'HGVs_4_or_more_rigid_axle', 'HGVs_3_or_4_articulated_axle',  
'HGVs_5_articulated_axle', 'HGVs_6_articulated_axle', 'all_HGVs',  
'all_motor_vehicles']
```

Perform the following preprocessing steps:

1. Create two new columns, 'month' and 'weekday', from the 'count_date' column. You may find the following functions useful:

- weekday:
<https://spark.apache.org/docs/4.1.0/api/python/reference/pyspark.sql/api/pyspark.sql.functions.weekday.html>
- month:
<https://spark.apache.org/docs/4.1.0/api/python/reference/pyspark.sql/api/pyspark.sql.functions.month.html>

2. Convert all values in the column 'direction_of_travel' to uppercase

3. Drop the rows where 'all_motor_vehicles' is NULL. You may find this dropna function useful: <https://spark.apache.org/docs/latest/api/python/reference/pyspark.sql/api/pyspark.sql.DataFrame.dropna.html>.

4. Use the one-hot encoder

(<https://spark.apache.org/docs/4.1.0/api/python/reference/api/pyspark.ml.feature.OneHotEncoder.html>) to

encode the categorical features:

Python

```
['direction_of_travel', 'hour', 'region_ons_code', 'local_authority_code',  
'month', 'weekday']
```

5. Ensure the following numerical features are of type double, and **standardize** them using [StandardScaler](#):

Python

```
['latitude', 'longitude']
```

6. Split the data by year as follows, and report the sample size for each set:

- Training: 2000–2021
- Validation: 2022–2023
- Testing: 2024

[3 marks]

Note: Do not use any vehicle subtype counts (i.e., 'pedal_cycles', 'two_wheeled_motor_vehicles', 'cars_and_taxis', 'buses_and_coaches', 'LGVs', 'HGVs_2_rigid_axle', 'HGVs_3_rigid_axle', 'HGVs_4_or_more_rigid_axle', 'HGVs_3_or_4_articulated_axle', 'HGVs_5_articulated_axle', 'HGVs_6_articulated_axle', 'all_HGVs') as features, as these directly contribute to the target variable.

B. Poisson regression

Use the processed features:

Python

```
['direction_of_travel', 'hour', 'region_ons_code', 'local_authority_code',  
'month', 'weekday', 'latitude', 'longitude']
```

to predict: 'all_motor_vehicles'.

Tune the hyperparameter `regParam` over [0.001, 0.01, 0.1, 1, 10, 100, 1000]. For each candidate value of `regParam`, create 5 random subsets by sampling 80% of the training set. Use the **last five digits** of your student registration number as the starting seed to generate five seeds. For example, if your student registration number is 111111111, use 11111, 11112, 11113, 11114, and 11115 as the random seeds. Please include your student registration number in both your code comments and your report.

For each sampled subset, train the model on that subset and evaluate it on the validation set. Then, for each value of `regParam`, compute the mean and standard deviation of the 5 validation MSE values. Plot the mean validation MSE against `regParam`, with error bars showing $\pm$ the standard deviation. Include the validation curve in your report and discuss the optimal value of `regParam`.

Retrain the Poisson regression model on the combined training and validation sets using the optimal `regParam` obtained in the previous step. Evaluate the final model on the test set and report the test MSE, along with the learned model coefficients.

Hint: Saving results to a separate file (e.g., .csv or .txt) instead of printing them, or setting the Spark log level to ERROR using [setLogLevel](#), can make it easier to locate the results to report and reduce the size of the generated log file. The same applies to Task C.

[3 marks]

C. Logistic regression

Using the training set only, compute the median value of 'all_motor_vehicles'.

Create a new binary target column 'traffic' for all instances as follows:

- traffic = 0 (low traffic) if all_motor_vehicles ≤ median
- traffic = 1 (high traffic) if all_motor_vehicles > median

Use the same processed features as in Task B to train a logistic regression model to predict 'traffic'.

Tune the hyperparameters `regParam` and `elasticNetParam` over [0.001, 0.01, 0.1, 1, 10, 100, 1000] and [0.0, 0.2, 0.5, 0.8, 1.0], respectively. Following the same setting as in task B, create 5 random training subsets, each obtained by sampling 80% of the training instances.

For each combination of `regParam` and `elasticNetParam`, train the model on the sampled training subsets and evaluate it on the validation set. Compute the mean validation accuracy and standard deviation across the 5 runs.

Plot the mean validation accuracy for all combinations of `regParam` and `elasticNetParam` (e.g., a grouped line plot), with error bars representing ± the standard deviation. Include the validation curve in the report and discuss the optimal values of `regParam` and `elasticNetParam`.

Retrain the logistic regression model on the combined training and validation sets using the optimal `regParam` and `elasticNetParam` obtained in the previous step. Evaluate the final model on the testing set and report the test accuracy along with the learned model coefficients.

[3 marks]

D. Select one observation or finding from Tasks A–C that you find interesting (e.g., model performance or feature importance/coefficients) and briefly discuss it.

[1 mark]

